

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа

Выполнила:

Шакина Анна

Группа

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Привязать клиентскую часть сайта к внешнему API средствами axios. Также необходимо реализовать моковое API с помощью JSON Server и подключить к нему авторизацию.

Ход работы

1. Подключение внешнего API и создание отдельного API-слоя

На втором этапе в проект был добавлен отдельный файл api.js, в котором был реализован централизованный слой для работы с внешним API. В данном файле был создан экземпляр axios с базовым адресом `http://localhost:3000`, а также настроен interceptor, автоматически добавляющий токен из `localStorage` в заголовок `Authorization`. Кроме того, были реализованы методы для основных операций: вход, регистрация, получение мероприятий, пользователей и регистраций, а также создание, изменение и удаление сущностей. Таким образом, проект был переведен на работу с mock API средствами JSON Server.

2. Обновление HTML-страниц и подключение axios

Далее были изменены основные HTML-страницы проекта. В них была подключена библиотека axios через CDN, а также добавлен файл `js/api.js`, который стал использоваться всеми страницами сайта. Такой подход позволил обеспечить единый способ обращения к внешнему API из разных частей проекта. В частности, новые подключения были добавлены на страницы входа, личного кабинета, страницы учителя и административной панели.

3. Перевод авторизации на работу через API

На следующем этапе была переработана логика авторизации. Если в исходной версии проекта вход выполнялся на основе локальных данных, то после доработки авторизация стала происходить через API-вызовы. Для этого страница входа была адаптирована под использование методов `login` и `register` из нового API-слоя. После успешного входа в локальное хранилище сохраняются токен доступа и информация о текущем пользователе, а маршрутизация по ролям выполняется уже на основе этих данных. Кроме того, была обновлена логика сессии: теперь защищенные

страницы зависят не только от наличия пользователя, но и от наличия accessToken.

4. Перевод пользовательских страниц на получение данных через API

После этого была переработана логика основных пользовательских страниц. Главная страница стала получать список мероприятий через `api.getEvents()`, а не из локального хранилища. При этом были сохранены ранее реализованные элементы интерфейса: карусель ближайших мероприятий, поиск, фильтрация по категориям и месту, а также карточки событий. В новой версии данные сначала загружаются через API, затем нормализуются и отображаются на странице.

Аналогичным образом была переработана страница отдельного мероприятия. В новой версии на ней используется загрузка данных о мероприятии и регистрациях через API, а запись и отмена записи выполняются не локально, а через HTTP-запросы. При этом была сохранена логика, согласно которой записываться на мероприятие может только пользователь с ролью ученика.

Также были адаптированы личный кабинет ученика и кабинет учителя. В этих разделах данные о мероприятиях, регистрациях и пользователях теперь загружаются через методы API, а затем используются для построения интерфейса. Таким образом, личные кабинеты перестали зависеть от локальных массивов данных и были переведены на клиент-серверную модель взаимодействия.

5. Административная панель

Отдельным этапом была переработана административная панель. В ходе лабораторной работы в ней был реализован переход на работу с внешним API при создании, редактировании и удалении пользователей и мероприятий. Для этого были использованы методы нового API-слоя, а загрузка основных сущностей стала выполняться асинхронно. В результате административная панель стала работать не с локальными структурами данных, а с `mock API`, что позволило приблизить проект к реальной клиент-серверной архитектуре.

Вывод

В результате выполнения лабораторной работы сайт мероприятий был успешно адаптирован для работы с внешним API. Если в исходной версии проект использовал локальные данные и тестовых пользователей, то в новой версии был добавлен отдельный API-слой на axios, подключен mock API на JSON Server. Основные страницы проекта – вход, список мероприятий, страница мероприятия, личный кабинет ученика, кабинет учителя и административная панель – были переведены на работу через HTTP-запросы. Следовательно, поставленная цель лабораторной работы была достигнута: проект стал ближе к реальной клиент-серверной архитектуре и перестал быть полностью зависимым от локального хранилища браузера.